Python Code Summarization with Pretrained Transformers

T5-small, BART-base, PLBART-base and CodeT5-base are fine-tuned to write a docstring for a Python function, on the CodeXGLUE code-to-text dataset.

Warning: about 25 % of the inputs still contain the target docstring, so all scores are inflated. Runs A/B and run C use different data sizes and must not be compared directly.

All figures, tables and numbers in this report are taken directly from the stored outputs of notebooks/code_summarization_transformers.ipynb.

Execution: real GPU runs. Runs A and B are partial by design: 4 epochs on 10,072 samples (first 4 %), tested on 298. Run C: 1 epoch on 176,274 samples (first 70 %), tested on 7,459.

Test-set results (BLEU 0-100, ROUGE 0-1)

Run	Model	Train	Ep	Test	Loss	BLEU	R-1	R-2	R-L
A	T5-small	10072	4	298	1.347	5.19	0.336	0.215	0.317
A	PLBART-base	10072	4	298	1.370	5.99	0.354	0.227	0.333
B	BART-base	10072	4	298	1.192	5.63	0.344	0.211	0.325
B	CodeT5	10072	4	298	1.036	10.24	0.408	0.259	0.381
C	CodeT5	176274	1	7459	1.016	7.60	0.403	0.256	0.378

Predictions on code from outside the dataset (first line)

```
Input: def binary_search(nums, target), an iterative binary search (not in the dataset)

A T5-small     Define binary_search(nums, target): left, right = 0, len(nums) - 1 while left = right...
A PLBART-base  return -1 if no binary search
B BART-base    Perform binary search on a list of integers.
B CodeT5       Returns the index of the binary search.
C CodeT5       Binary search for a given number.
```

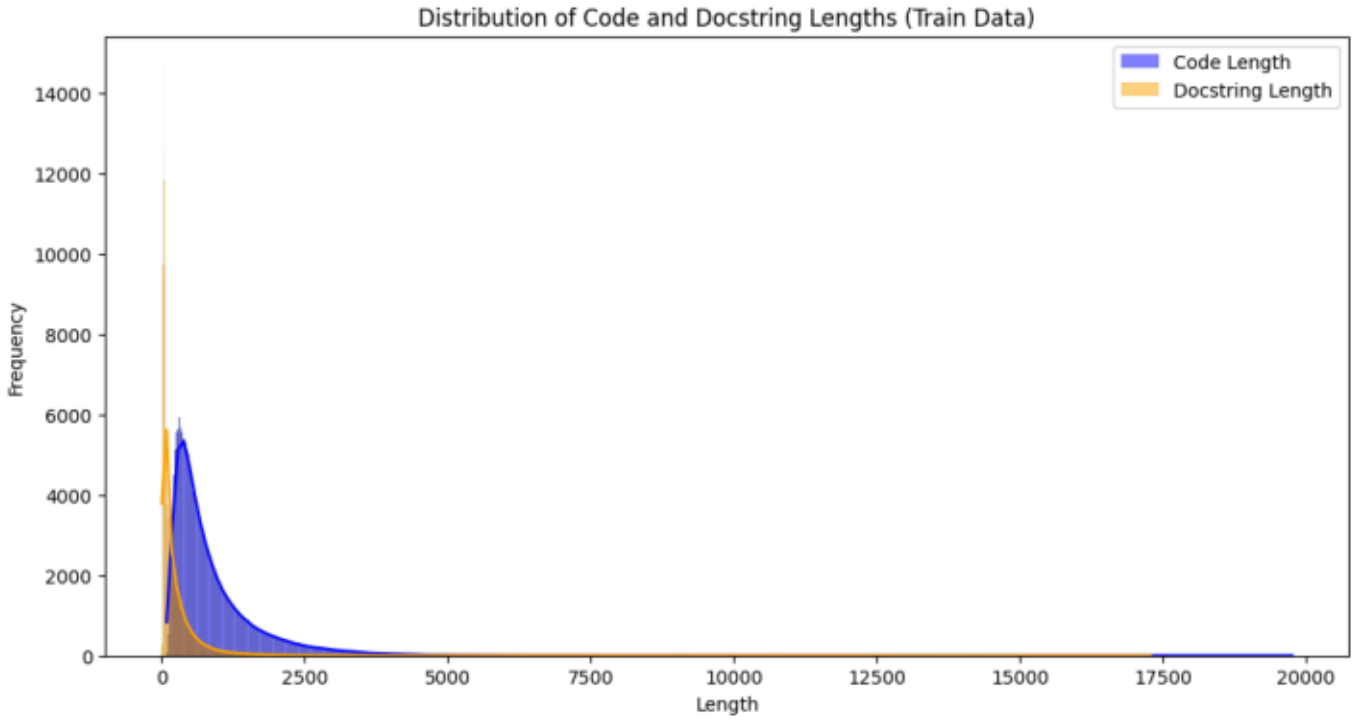


Figure 1. Code and docstring length distributions in the training data.

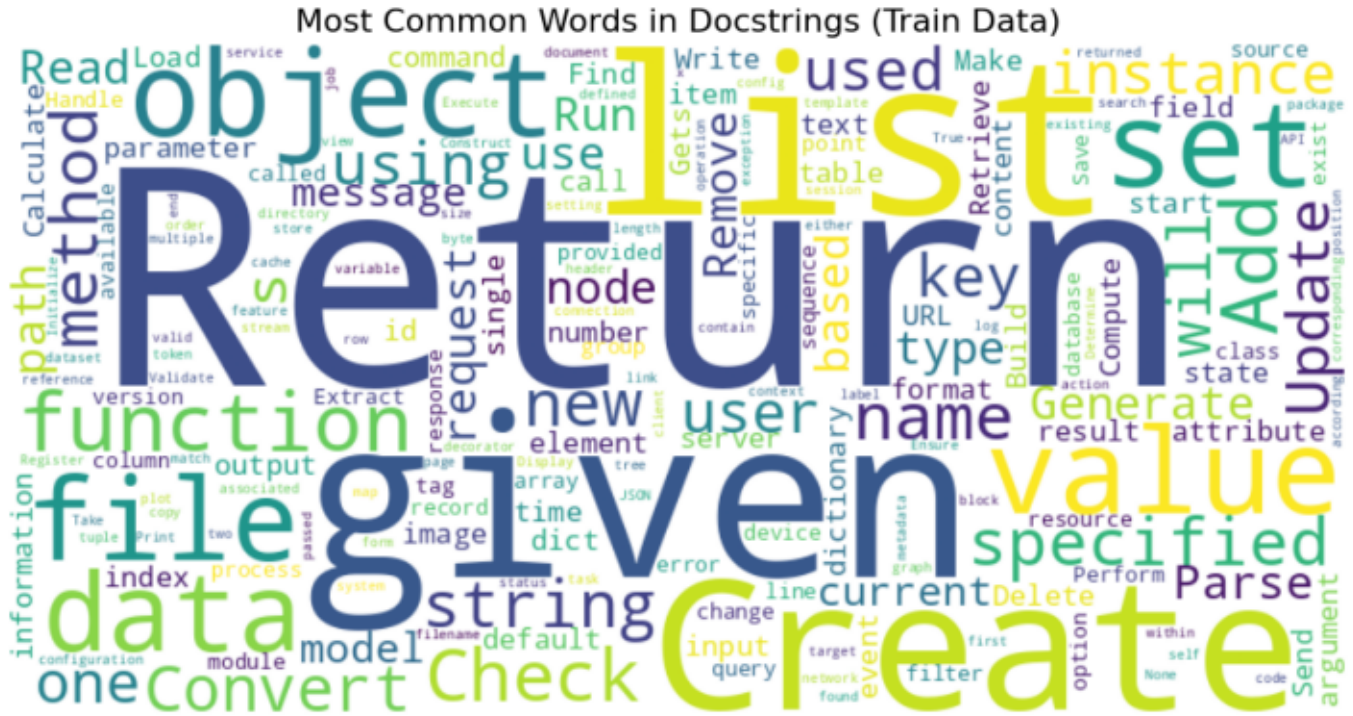


Figure 2. Most common words in the training docstrings.

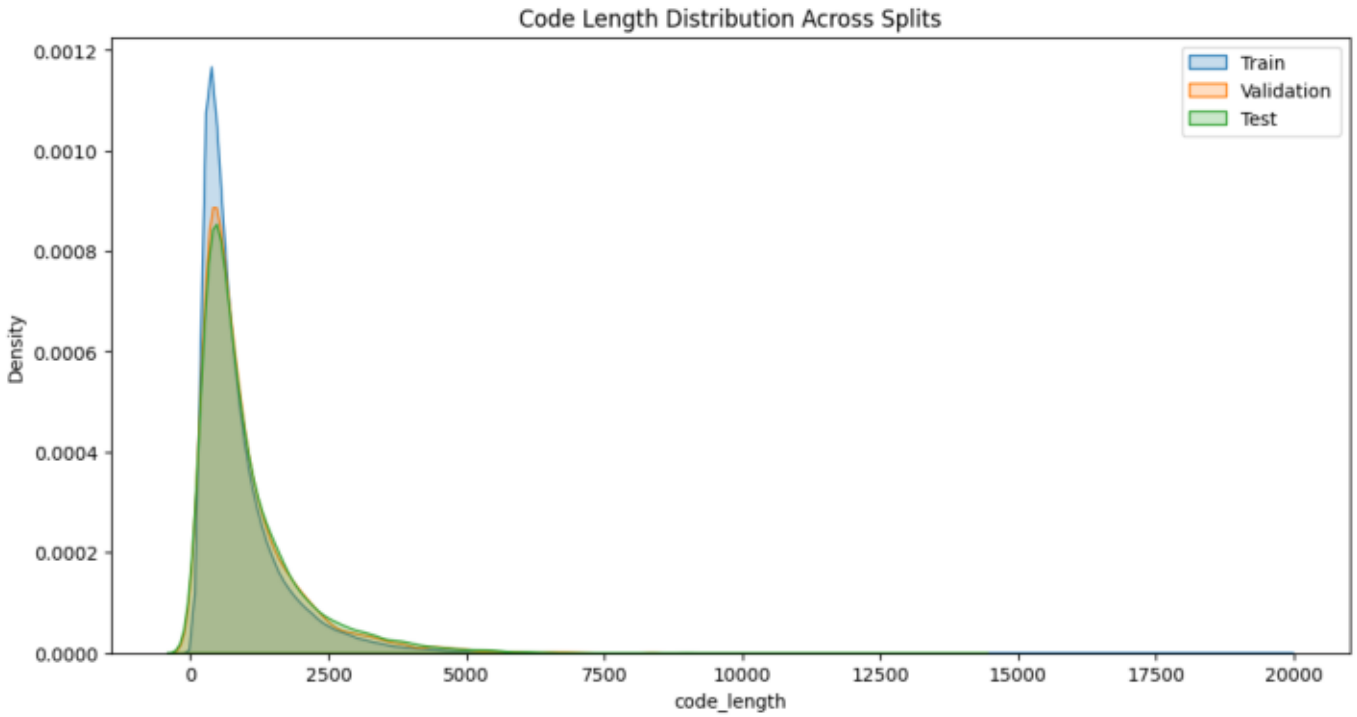


Figure 3. Code length distributions across the splits.

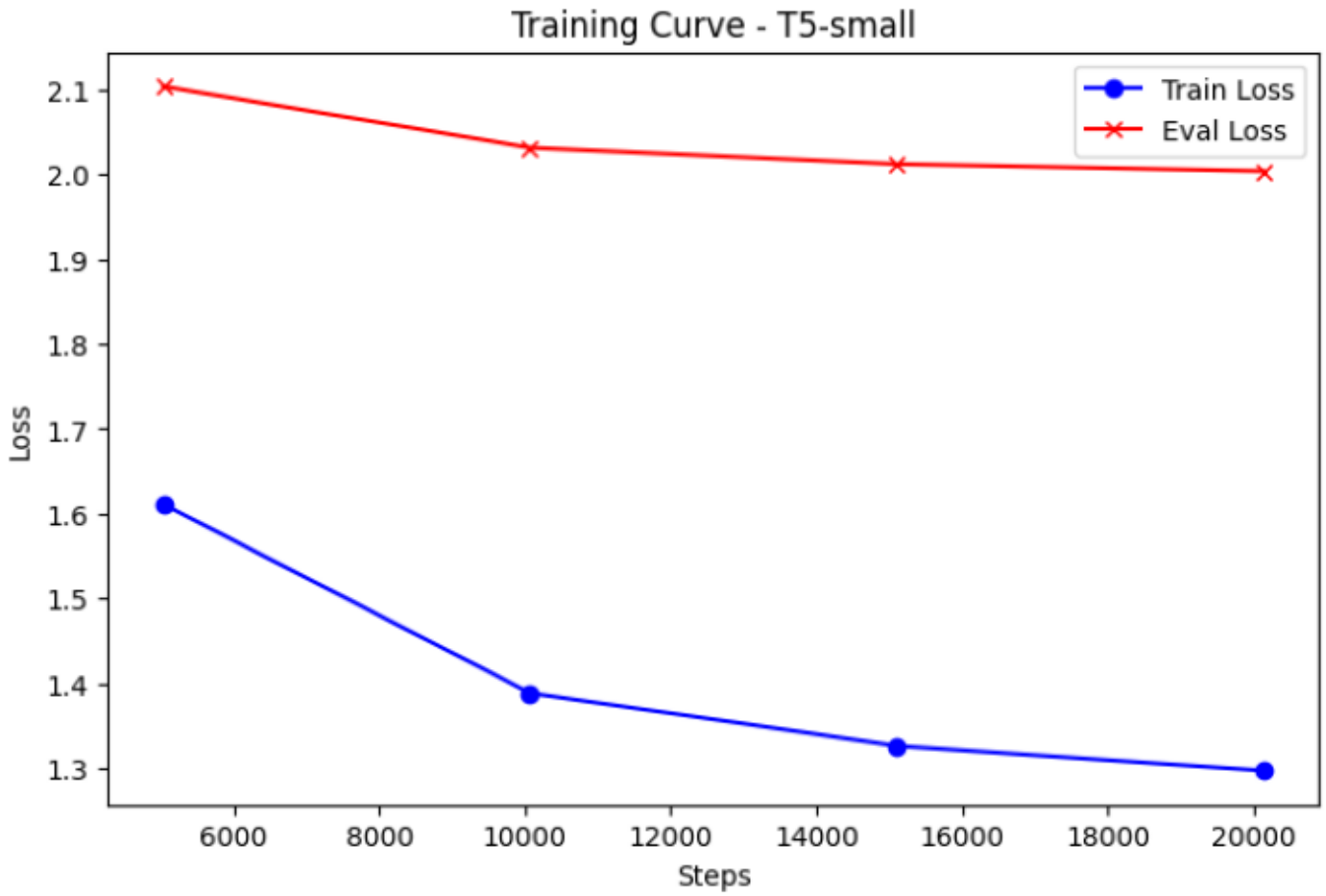


Figure 4. Training curve of T5-small (run A).

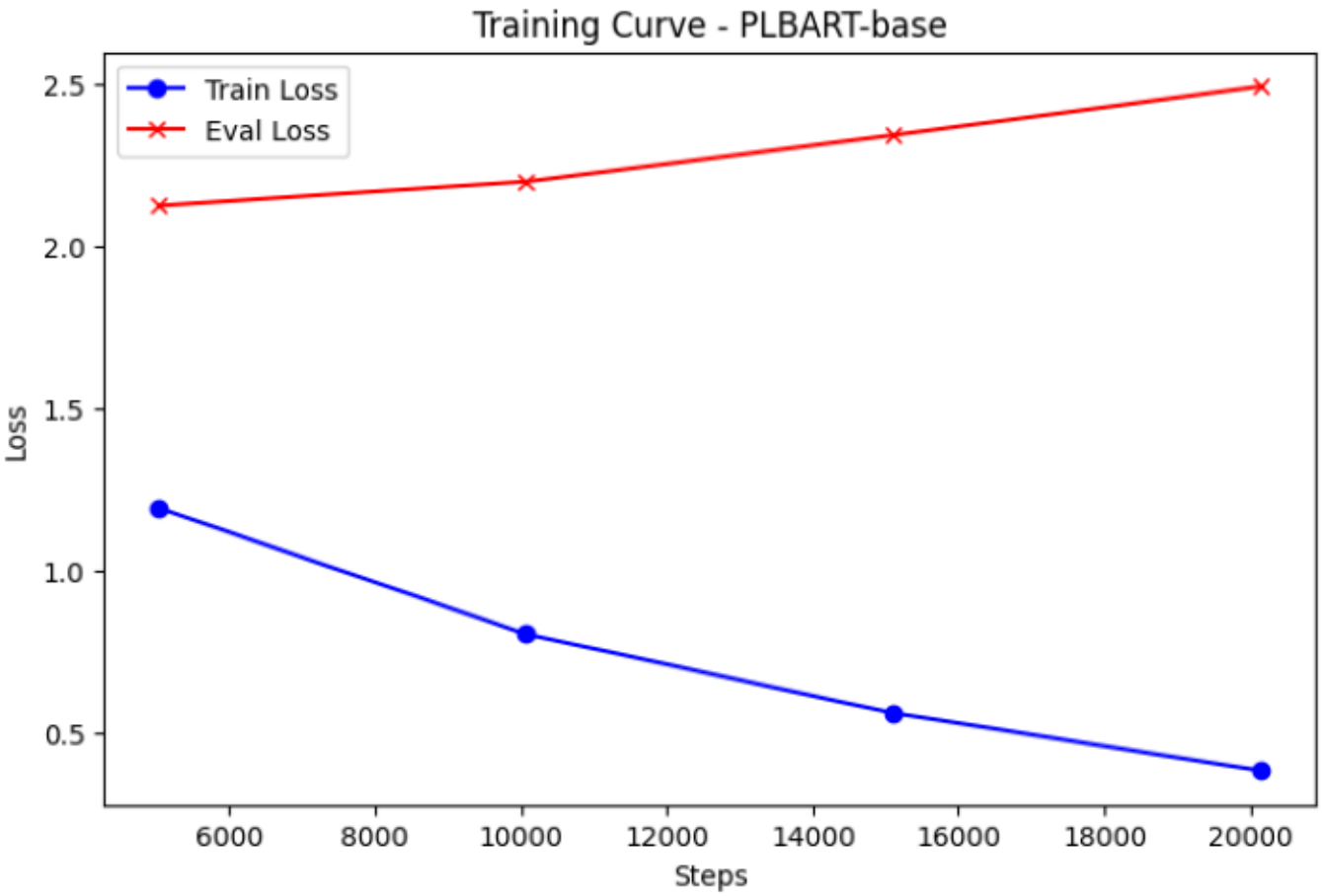


Figure 5. Training curve of PLBART-base (run A).

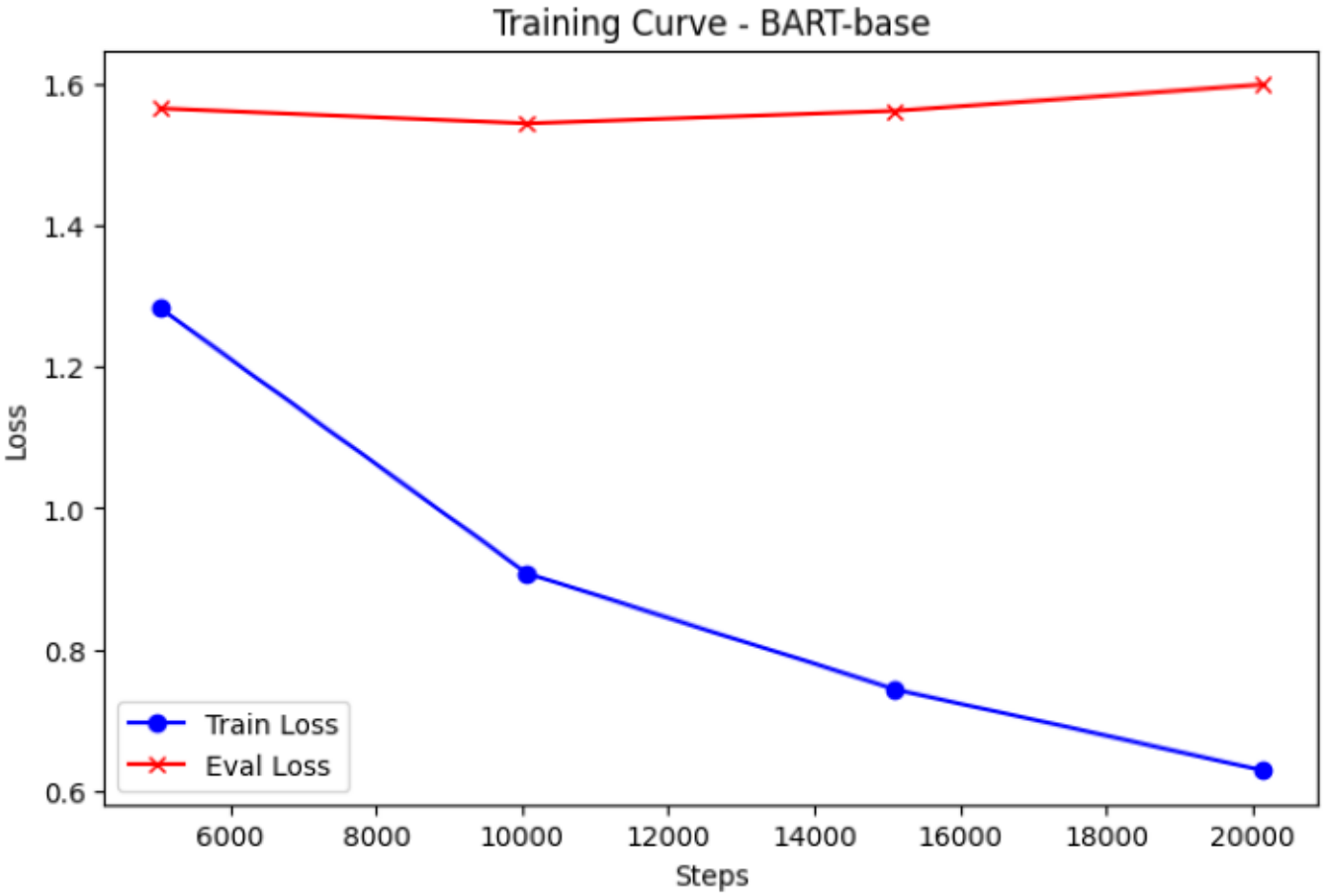


Figure 6. Training curve of BART-base (run B).

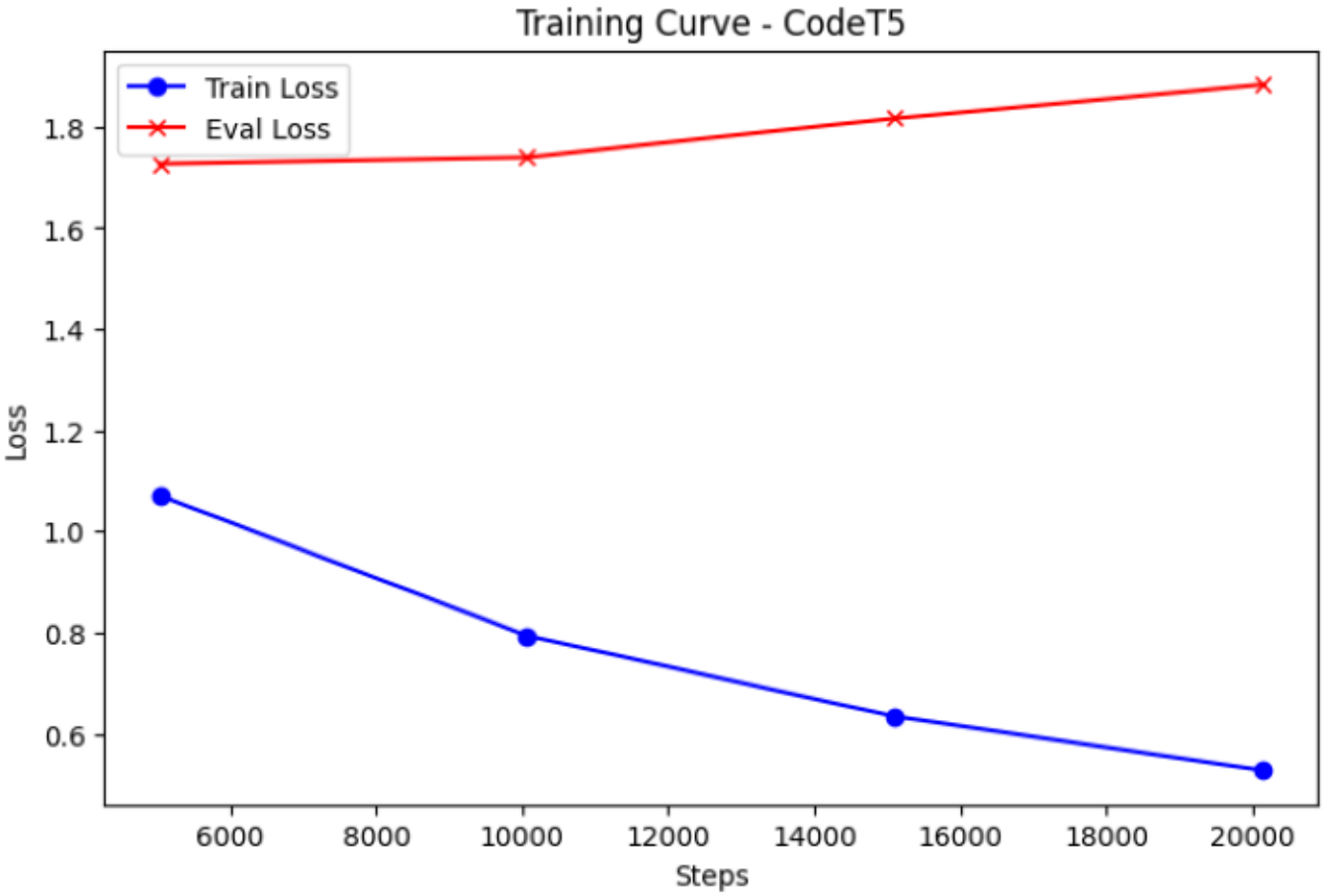


Figure 7. Training curve of CodeT5 (run B).

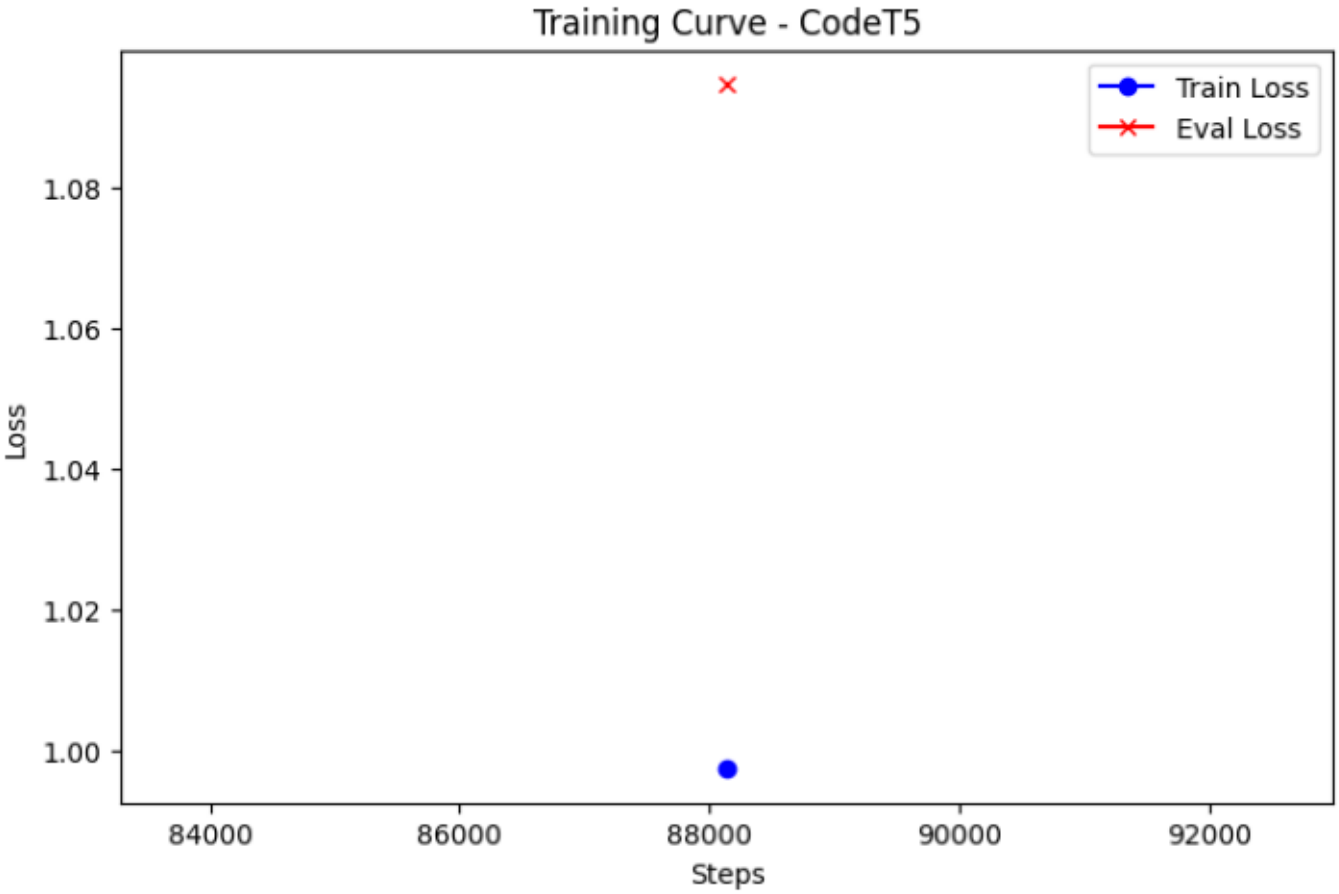


Figure 8. Training curve of CodeT5 (run C, one epoch on 176,274 samples).